

第46章 电源管理—实现低功耗

本章参考数据：《STM32F4xx 参考手册》、《STM32F4xx 规格书》、库说明文档《stm32f4xx_dsp_stdperiph_lib_um.chm》。

46.1 STM32 的电源管理简介

电源对电子设备的重要性不言而喻，它是保证系统稳定运行的基础，而保证系统能稳定运行后，又有低功耗的要求。在很多应用场合中都对电子设备的功耗要求非常苛刻，如某些传感器信息采集设备，仅靠小型的电池提供电源，要求工作长达数年之久，且期间不需要任何维护；由于智慧穿戴设备的小型化要求，电池体积不能太大导致容量也比较小，所以也很有必要从控制功耗入手，提高设备的续行时间。因此，STM32 有专门的电源管理外设监控电源并管理设备的运行模式，确保系统正常运行，并尽量降低器件的功耗。

46.1.1 电源监控器

STM32 芯片主要通过引脚 VDD 从外部获取电源，在它的内部具有电源监控器用于检测 VDD 的电压，以实现复位功能及掉电紧急处理功能，保证系统可靠地运行。

1. 上电复位与掉电复位(POR 与 PDR)

当检测到 VDD 的电压低于阈值 VPOR 及 VPDR 时，无需外部电路辅助，STM32 芯片会自动保持在复位状态，防止因电压不足强行工作而带来严重的后果。见图 46-1，在刚开始电压低于 VPOR 时(约 1.72V)，STM32 保持在上电复位状态(POR, Power On Reset)，当 VDD 电压持续上升至大于 VPOR 时，芯片开始正常运行，而在芯片正常运行时候，当检测到 VDD 电压下降至低于 VPDR 阈值(约 1.68V)，会进入掉电复位状态(PDR, Power Down Reset)。

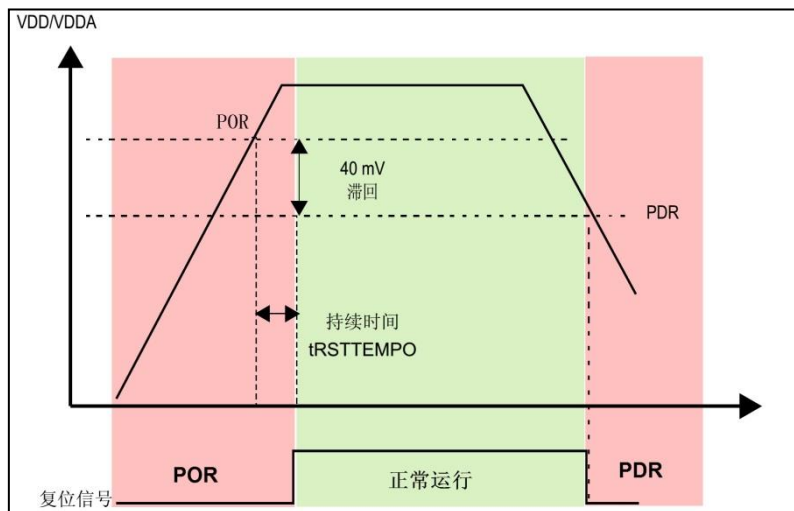


图 46-1 POR 与 PDR

2. 欠压复位(BOR)

POR 与 PDR 的复位电压阈值是固定的, 如果用户想要自行设定复位阈值, 可以使用 STM32 的 BOR 功能(Brownout Reset)。它可以编程控制电压检测工作在表 46-1 中的阈值级别, 通过修改“选项字节”(某些特殊寄存器)中的 BOR_LEV 位即可控制阈值级别。其复位控制示意图见图 46-2。

表 46-1 BOR 欠压阈值等级

等级	条件	电压值
1 级欠压阈值	下降沿	2.19V
	上升沿	2.29V
2 级欠压阈值	下降沿	2.50V
	上升沿	2.59V
3 级欠压阈值	下降沿	2.83V
	上升沿	2.92V

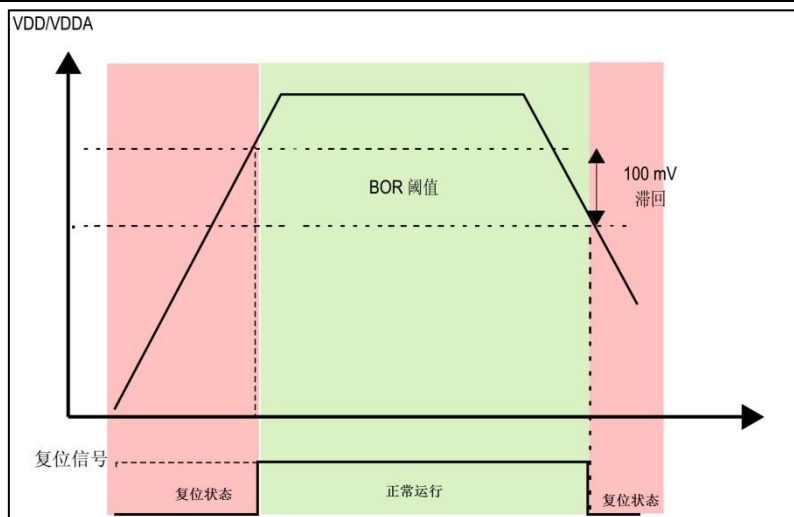


图 46-2 BOR 复位控制

3. 可编程电压检测器 PVD

上述 POR、PDR 以及 BOR 功能都是使用其电压阈值与外部供电电压 VDD 比较, 当低于工作阈值时, 会直接进入复位状态, 这可防止电压不足导致的误操作。除此之外, STM32 还提供了可编程电压检测器 PVD, 它也是实时检测 VDD 的电压, 当检测到电压低于 VPVD 阈值时, 会向内核产生一个 PVD 中断(EXTI16 线中断)以使内核在复位前进行紧急处理。该电压阈值可通过电源控制寄存器 PWR_CSR 设置。

使用 PVD 可配置 8 个等级, 见表 46-2。其中的上升沿和下降沿分别表示类似图 46-2 中 VDD 电压上升过程及下降过程的阈值。

表 46-2 PVD 的阈值等级

阈值等级	条件	最小值	典型值	最大值	单位
级别 0	上升沿	2.09	2.14	2.19	V
	下降沿	1.98	2.04	2.08	V
级别 1	上升沿	2.23	2.3	2.37	V
	下降沿	2.13	2.19	2.25	V

46.1.2 STM32 的电源系统

□ 备份域电路

STM32 的 LSE 振荡器、RTC、备份寄存器及备份 SRAM 这些器件被包含进备份域电路中，这部分的电路可以通过 STM32 的 VBAT 引脚获取供电电源，在实际应用中一般会使用 3V 的钮扣电池对该引脚供电。

在图中备份域电路的左侧有一个电源开关结构，它的功能类似图 46-4 中的双二极管，在它的上方连接了 VBAT 电源，下方连接了 VDD 主电源(一般为 3.3V)，右侧引出到备份域电路中。当 VDD 主电源存在时，由于 VDD 电压较高，备份域电路通过 VDD 供电，当 VDD 掉电时，备份域电路由钮扣电池通过 VBAT 供电，保证电路能持续运行，从而可利用它保留关键数据。

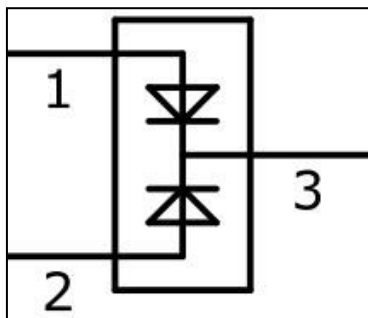


图 46-4 双二极管结构

□ 调压器供电电路

在 STM32 的电源系统中调压器供电的电路是最主要的部分，调压器为备份域及待机电路以外的所有数字电路供电，其中包括内核、数字外设以及 RAM，调压器的输出电压约为 1.2V，因而使用调压器供电的这些电路区域被称为 1.2V 域。

调压器可以运行在“运行模式”、“停止模式”以及“待机模式”。在运行模式下，1.2V 域全功率运行；在停止模式下 1.2V 域运行在低功耗状态，1.2V 区域的所有时钟都被关闭，相应的外设都停止了工作，但它会保留内核寄存器以及 SRAM 的内容；在待机模式下，整个 1.2V 域都断电，该区域的内核寄存器及 SRAM 内容都会丢失(备份区域的寄存器及 SRAM 不受影响)。

□ ADC 电源及参考电压

为了提高转换精度，STM32 的 ADC 配有独立的电源接口，方便进行单独的滤波。ADC 的工作电源使用 VDDA 引脚输入，使用 VSSA 作为独立的地连接，VREF 引脚则为 ADC 提供测量使用的参考电压。

46.1.3 STM32 的功耗模式

按功耗由高到低排列，STM32 具有运行、睡眠、停止和待机四种工作模式。上电复位后 STM32 处于运行状态时，当内核不需要继续运行，就可以选择进入后面的三种低功耗模式降低功耗，这三种模式中，电源消耗不同、唤醒时间不同、唤醒源不同，用户需要根据应用需求，选择最佳的低功耗模式。三种低功耗的模式说明见表 46-3。

表 46-3STM32 的低功耗模式说明

模式	说明	进入方式	唤醒方式	对 1.2V 区域时钟的影响	对 VDD 区域时钟	调压
----	----	------	------	----------------	------------	----

					的影响	器
睡眠	内核停止，所有外设包括 M4 核心的外设，如 NVIC、系统时钟(SysTick)等仍在运行	调用 WFI 命令	任一中断	内核时钟关，对其他时钟和 ADC 时钟无影响	无	开
		调用 WFE 命令	唤醒事件			
停止	所有的时钟都已停止	配置 PWR_CR 寄存器的 PDDS +LPDS 位 +SLEEPD EEP 位 +WFI 或 WFE 命令	任一外部中断(在外部中断寄存器中设置)	关闭所有 1.2V 区域的时钟	HSI 和 HSE 的振荡器关闭	开启或处于低功耗模式(依据电源控制寄存器的设定)
待机	1.2V 电源关闭	配置 PWR_CR 寄存器的 PDDS +SLEEPD EEP 位 +WFI 或 WFE 命令	WKUP 引脚的上升沿、RTC 闹钟事件、NRST 引脚上的外部复位、IWDG 复位			关

从表中可以看到，这三种低功耗模式层层递进，运行的时钟或芯片功能越来越少，因而功耗越来越低。

1. 睡眠模式

在睡眠模式中，仅关闭了内核时钟，内核停止运行，但其片上外设，CM4 核心的外设全都还照常运行。有两种方式进入睡眠模式，它的进入方式决定了从睡眠唤醒的方式，分别是 WFI(wait for interrupt)和 WFE(wait for event)，即由等待“中断”唤醒和由“事件”唤醒。睡眠模式的各种特性见表 46-4。

表 46-4 睡眠模式的各种特性

特性	说明
立即睡眠	在执行 WFI 或 WFE 指令时立即进入睡眠模式。
退出时睡眠	在退出优先级最低的中断服务程序后才进入睡眠模式。
进入方式	内核寄存器的 SLEEPDEEP = 0，然后调用 WFI 或 WFE 指令即可进入睡眠模式； 另外若内核寄存器的 SLEEPONEXIT=0 时，进入“立即睡眠”模式， SLEEPONEXIT=1 时，进入“退出时睡眠”模式。
唤醒方式	如果是使用 WFI 指令睡眠的，则可使用任意中断唤醒； 如果是使用 WFE 指令睡眠的，则由事件唤醒。
睡眠时	关闭内核时钟，内核停止，而外设正常运行，在软件上表现为不再执行新的代码。这个状态会保留睡眠前的内核寄存器、内存的数据。

唤醒延迟	无延迟。
唤醒后	若由中断唤醒，先进入中断，退出中断服务程序后，接着执行 WFI 指令后的程序；若由事件唤醒，直接接着执行 WFE 后的程序。

2. 停止模式

在停止模式中，进一步关闭了其它所有的时钟，于是所有的外设都停止了工作，但由于其 1.2V 区域的部分电源没有关闭，还保留了内核的寄存器、内存的信息，所以从停止模式唤醒，并重新开启时钟后，还可以从上次停止处继续执行代码。停止模式可以由任意一个外部中断(EXTI)唤醒。在停止模式可以选择电压调节器为开模式或低功耗模式，可选择内部 FLASH 工作在正常模式或掉电模式。停止模式的特性见表 46-5。

表 46-5 停止模式的特性

特性	说明
调压器低功耗模式	在停止模式下调压器可工作在正常模式或低功耗模式，可进一步降低功耗
FLASH 掉电模式	在停止模式下 FLASH 可工作在正常模式或掉电模式，可进一步降低功耗
进入方式	内核寄存器的 SLEEPDEEP=1，PWR_CR 寄存器中的 PDDS=0，然后调用 WFI 或 WFE 指令即可进入停止模式； PWR_CR 寄存器的 LPDS=0 时，调压器工作在正常模式，LPDS=1 时工作在低功耗模式； PWR_CR 寄存器的 FPDS=0 时，FLASH 工作在正常模式，FPDS=1 时进入掉电模式。
唤醒方式	如果是使用 WFI 指令睡眠的，可使用任意 EXTI 线的中断唤醒； 如果是使用 WFE 指令睡眠的，可使用任意配置为事件模式的 EXTI 线事件唤醒。
停止时	内核停止，片上外设也停止。这个状态会保留停止前的内核寄存器、内存的数据。
唤醒延迟	基础延迟为 HSI 振荡器的启动时间，若调压器工作在低功耗模式，还需要加上调压器从低功耗切换至正常模式下的时间，若 FLASH 工作在掉电模式，还需要加上 FLASH 从掉电模式唤醒的时间。
唤醒后	若由中断唤醒，先进入中断，退出中断服务程序后，接着执行 WFI 指令后的程序；若由事件唤醒，直接接着执行 WFE 后的程序。唤醒后，STM32 会使用 HIS 作为系统时钟。

3. 待机模式

待机模式，它除了关闭所有的时钟，还把 1.2V 区域的电源也完全关闭了，也就是说，从待机模式唤醒后，由于没有之前代码的运行记录，只能对芯片复位，重新检测 boot 条件，从头开始执行程序。它有四种唤醒方式，分别是 WKUP(PA0)引脚的上升沿，RTC 闹钟事件，NRST 引脚的复位和 IWDG(独立看门狗)复位。

表 46-6 待机模式的特性

特性	说明
进入方式	内核寄存器的 SLEEPDEEP=1，PWR_CR 寄存器中的 PDDS=1，PWR_CR 寄存器中的唤醒状态位 WUF=0，然后调用 WFI 或 WFE 指令即可进入待机模式；
唤醒方式	通过 WKUP 引脚的上升沿，RTC 闹钟、唤醒、入侵、时间戳事件或 NRST 引脚外部复位及 IWDG 复位唤醒。

待机时	内核停止，片上外设也停止；内核寄存器、内存的数据会丢失；除复位引脚、RTC_AF1 引脚及 WKUP 引脚，其它 I/O 口均工作在高阻态。
唤醒延迟	芯片复位的时间
唤醒后	相当于芯片复位，在程序表现为从头开始执行代码。

在以上讲解的睡眠模式、停止模式及待机模式中，若备份域电源正常供电，备份域内的 RTC 都可以正常运行、备份域内的寄存器及备份域内的 SRAM 数据会被保存，不受功耗模式影响。

46.2 电源管理相关的库函数及命令

STM32 标准库对电源管理提供了完善的函数及命令，使用它们可以方便地进行控制，本小节对这些内容进行讲解。

46.2.1 配置 PVD 监控功能

PVD 可监控 VDD 的电压，当它低于阈值时可产生 PVD 中断以让系统进行紧急处理，这个阈值可以直接使用库函数 PWR_PVDLevelConfig 配置成前面表 46-2 中说明的阈值等级。

46.2.2 WFI 与 WFE 命令

我们了解到进入各种低功耗模式时都需要调用 WFI 或 WFE 命令，它们实质上都是内核指令，在库文件 core_cmInstr.h 中把这些指令封装成了函数，见代码清单 25-1。

代码清单 46-1 WFI 与 WFE 的指令定义(core_cmInstr.h 文件)

```
1
2 /** \brief Wait For Interrupt
3
4     Wait For Interrupt is a hint instruction that suspends execution
5     until one of a number of events occurs.
6 */
7 #define __WFI                                __wfi
8
9
10 /** \brief Wait For Event
11
12     Wait For Event is a hint instruction that permits the processor to enter
13     a low-power state until one of a number of events occurs.
14 */
15 #define __WFE                                __wfe
```

对于这两个指令，我们应用时一般只需要知道，调用它们都能进入低功耗模式，需要使用函数的格式“__WFI();”和“__WFE();”来调用(因为__wfi及__wfe是编译器内置的函数，函数内部使用调用了相应的汇编指令)。其中 WFI 指令决定了它需要用中断唤醒，而 WFE 则决定了它可用事件来唤醒，关于它们更详细的区别可查阅《cortex-CM3/CM4 权威指南》了解。

46.2.3 进入停止模式

直接调用 WFI 和 WFE 指令可以进入睡眠模式，而进入停止模式则还需要在调用指令前设置一些寄存器位，STM32 标准库把这部分的操作封装到 PWR_EnterSTOPMode 函数中了，它的定义见代码清单 44-2。

代码清单 46-2 进入停止模式

```
1 /**
2  * @brief 进入停止模式
3  *
4  * @note 在停止模式下所有 I/O 的会保持在停止前的状态
5  * @note 从停止模式唤醒后，会使用 HSI 作为时钟源
6  * @note 调压器若工作在低功耗模式，可减少功耗，但唤醒时会增加延迟
7  * @param PWR_Regulator: 设置停止模式时调压器的工作模式
8  *          @arg PWR_MainRegulator_ON: 调压器正常运行
9  *          @arg PWR_LowPowerRegulator_ON: 调压器低功耗运行
10 * @param PWR_STOPEntry: 设置使用 WFI 还是 WFE 进入停止模式
11 *          @arg PWR_STOPEntry_WFI: WFI 进入停止模式
12 *          @arg PWR_STOPEntry_WFE: WFE 进入停止模式
13 * @retval None
14 */
15 void PWR_EnterSTOPMode(uint32_t PWR_Regulator, uint8_t PWR_STOPEntry)
16 {
17     uint32_t tmpreg = 0;
18
19     /* 设置调压器的模式 -----*/
20     tmpreg = PWR->CR;
21     /* 清除 PDDS 及 LPDS 位 */
22     tmpreg &= CR_DS_MASK;
23
24     /* 根据 PWR_Regulator 的值(调压器工作模式)配置 LPDS, MRLVDS 及 LPLVDS 位 */
25     tmpreg |= PWR_Regulator;
26
27     /* 写入参数值到寄存器 */
28     PWR->CR = tmpreg;
29
30     /* 设置内核寄存器的 SLEEPDEEP 位 */
31     SCB->SCR |= SCB_SCR_SLEEPDEEP_Msk;
32
33     /* 设置进入停止模式的方式-----*/
34     if (PWR_STOPEntry == PWR_STOPEntry_WFI) {
35         /* 需要中断唤醒*/
36         __WFI();
37     } else {
38         /* 需要事件唤醒 */
39         __WFE();
40     }
41     /* 以下的程序是当重新唤醒时才执行的，清除 SLEEPDEEP 位的状态*/
42     SCB->SCR &= (uint32_t)~((uint32_t)SCB_SCR_SLEEPDEEP_Msk);
43 }
```

这个函数有两个输入参数，分别用于控制调压器的模式及选择使用 WFI 或 WFE 停止，代码中先是根据调压器的模式配置 PWR_CR 寄存器，再把内核寄存器的 SLEEPDEEP 位置 1，这样再调用 WFI 或 WFE 命令时，STM32 就不是睡眠，而是进入停止模式了。函数结尾处的语句用于复位 SLEEPDEEP 位的状态，由于它是在 WFI 及 WFE 指令之后的，所以这部分代码是在 STM32 被唤醒的时候才会执行。

要注意的是进入停止模式后，STM32 的所有 I/O 都保持在停止前的状态，而当它被唤醒时，STM32 使用 HSI 作为系统时钟(16MHz)运行，由于系统时钟会影响很多外设的工作状态，所以一般我们在唤醒后会重新开启 HSE，把系统时钟设置会原来的状态。

前面提到在停止模式中还可以控制内部 FLASH 的供电，控制 FLASH 是进入掉电状态还是正常供电状态，这可以使用库函数 PWR_FlashPowerDownCmd 配置，它其实只是封装了一个对 FPDS 寄存器位操作的语句，见代码清单 46-3。这个函数需要在进入停止模式前被调用，即应用时需要把它放在上面的 PWR_EnterSTOPMode 之前。

代码清单 46-3 控制 FLASH 的供电状态

```
1 /**
2  * @brief 设置内部 FLASH 在停止模式时是否工作在掉电状态
3  *        掉电状态可使功耗更低，但唤醒时会增加延迟
4  * @param NewState:
5  *        ENABLE:FLASH 掉电
6  *        DISABLE:FLASH 正常运行
7  * @retval None
8  */
9 void PWR_FlashPowerDownCmd(FunctionalState NewState)
10 {
11     /*配置 FPDS 寄存器位*/
12     *(__IO uint32_t *) CR_FPDS_BB = (uint32_t)NewState;
13 }
```

46.2.4 进入待机模式

类似地，STM32 标准库也提供了控制进入待机模式的函数，其定义见代码清单 44-3。

代码清单 46-4 进入待机模式

```
1 /**
2  * @brief 进入待机模式
3  * @note  待机模式时，除以下引脚，其余引脚都在高阻态：
4  *        -复位引脚
5  *        - RTC_AF1 引脚 (PC13) (需要使能侵入检测、时间戳事件或 RTC 闹钟事件)
6  *        - RTC_AF2 引脚 (PI8) (需要使能侵入检测或时间戳事件)
7  *        - WKUP 引脚 (PA0) (需要使能 WKUP 唤醒功能)
8  * @note 在调用本函数前还需要清除 WUF 寄存器位
9  * @param None
10 * @retval None
11 */
12 void PWR_EnterSTANDBYMode(void)
13 {
14     /* 选择待机模式 */
15     PWR->CR |= PWR_CR_PDDS;
16
17     /* 设置内核寄存器的 SLEEPDEEP 位 */
18     SCB->SCR |= SCB_SCR_SLEEPDEEP_Msk;
19
20     /* 存储操作完毕时才能进入待机模式，使用以下语句确保存储操作执行完毕 */
21     __force_stores();
22
23     /* 等待中断唤醒 */
24     __WFI();
25 }
26 }
```

该函数中先配置了 PDDS 寄存器位及 SLEEPDEEP 寄存器位，接着调用 __force_stores 函数确存储操作完毕后再调用 WFI 指令，从而进入待机模式。这里值得注意的是，待机模式也可以使用 WFE 指令进入的，如果您有需要可以自行修改；另外，由于这个函数没有操作 WUF 寄存器位，所以在实际应用中，调用本函数前，还需要清空 WUF 寄存器位才能进入待机模式。

在进入待机模式后，除了被使能了的用于唤醒的 I/O，其余 I/O 都进入高阻态，而从待机模式唤醒后，相当于复位 STM32 芯片，程序重新从头开始执行。

46.3 PWR—睡眠模式实验

在本小节中，我们以实验的形式讲解如何控制 STM32 进入低功耗睡眠模式。

46.3.1 硬件设计

实验中的硬件主要使用到了按键、LED 彩灯以及使用串口输出调试信息，这些硬件都与前面相应实验中的一致，涉及到硬件设计的可参考原理图或前面章节中的内容。

46.3.2 软件设计

本小节讲解的是“PWR—睡眠模式”实验，请打开配套的代码工程阅读理解。

1. 程序设计要点

- (1) 初始化用于唤醒的中断按键；
- (2) 进入睡眠状态；
- (3) 使用按键中断唤醒芯片；

2. 代码分析

main 函数

睡眠模式的程序比较简单，我们直接阅读它的 main 函数了解执行流程，见代码清单 25-2。

代码清单 46-5 睡眠模式的 main 函数(main.c 文件)

```
1
2 /**
3  * @brief 主函数
4  * @param 无
5  * @retval 无
6  */
7 int main(void)
8 {
9
10     LED_GPIO_Config();
11
12     /*初始化 USART1*/
13     Debug_USART_Config();
14
15     /* 初始化按键为中断模式，按下中断后会进入中断服务函数 */
16     EXTI_Key_Config();
```

```
17
18     printf("\r\n 欢迎使用野火  STM32  F407  开发板。 \r\n");
19     printf("\r\n 野火 F407  睡眠模式例程\r\n");
20
21     printf("\r\n 实验说明: \r\n");
22
23     printf("\r\n 1.本程序中, 绿灯表示 STM32 正常运行, 红灯表示睡眠状态, 蓝灯表示刚从睡眠状态被唤醒\r\n");
24     printf("\r\n 2.程序运行一段时间后自动进入睡眠状态, 在睡眠状态下, 可使用 KEY1 或 KEY2 唤醒\r\n");
25     printf("\r\n 3.本实验执行这样一个循环: \r\n");
26     printf("\r\n --》亮绿灯 (正常运行)->亮红灯 (睡眠模式)->按 KEY1 或 KEY2 唤醒->亮蓝灯 (刚被唤醒)--》 \r\n");
27     printf("\r\n 4.在睡眠状态下, DAP 下载器无法给 STM32 下载程序, \r\n");
28     printf("\r\n 可按 KEY1、KEY2 唤醒后下载, \r\n");
29     printf("\r\n 或按复位键使芯片处于复位状态, 然后在电脑上点击下载按钮, 再释放复位按键, 即可下载\r\n");
30
31     while (1) {
32         /*****执行任务*****/
33         printf("\r\n STM32 正常运行, 亮绿灯\r\n");
34
35         LED_GREEN;
36         Delay(0x3FFFFFFF);
37
38         /*****任务执行完毕, 进入睡眠降低功耗*****/
39
40
41         printf("\r\n 进入睡眠模式, 按 KEY1 或 KEY2 按键可唤醒\r\n");
42
43         //使用红灯指示, 进入睡眠状态
44         LED_RED;
45         //进入睡眠模式
46         __WFI(); //WFI 指令进入睡眠
47
48         //等待中断唤醒  K1 或 K2 按键中断
49
50         /***被唤醒, 亮蓝灯指示***/
51         LED_BLUE;
52         Delay(0x1FFFFFFF);
53
54         printf("\r\n 已退出睡眠模式\r\n");
55         //继续执行 while 循环
56     }
57 }
```

这个 main 函数的执行流程见图 46-5。

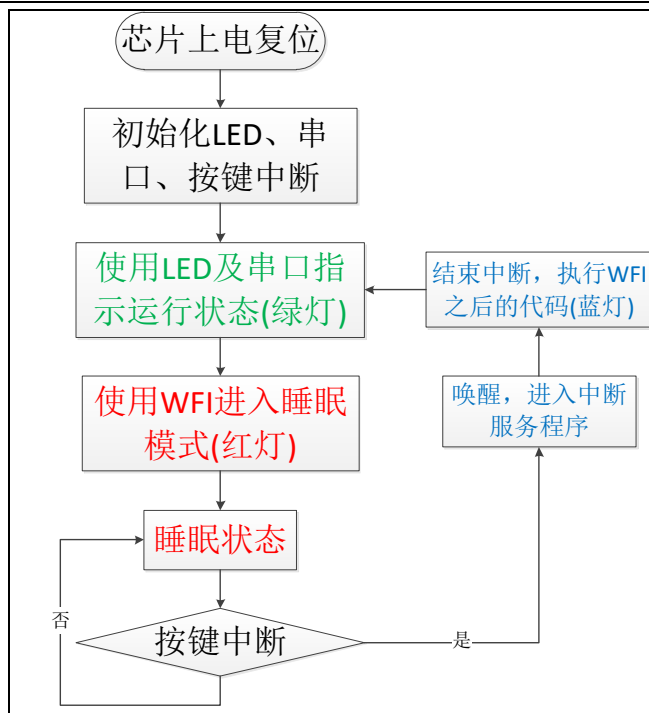


图 46-5 睡眠模式实验流程图

- (1) 程序中首先初始化了 LED 灯及串口以便用于指示芯片的运行状态，并且把实验板上的两个按键都初始化成了中断模式，以便当系统进入睡眠模式的时候可以通过按键来唤醒。这些硬件的初始化过程都跟前面章节中的一模一样。
- (2) 初始化完成后使用 LED 及串口表示运行状态，在本实验中，LED 彩灯为绿色时表示正常运行，红灯时表示睡眠状态，蓝灯时表示刚从睡眠状态中被唤醒。
- (3) 程序执行一段时间后，直接使用 WFI 指令进入睡眠模式，由于 WFI 睡眠模式可以使用任意中断唤醒，所以我们可以使用按键中断唤醒。
- (4) 当系统进入停止状态后，我们按下实验板上的 KEY1 或 KEY2 按键，即可使系统回到正常运行的状态，当执行完中断服务函数后，会继续执行 WFI 指令后的代码。

中断服务函数

系统刚被唤醒时会进入中断服务函数，见代码清单 25-3。

代码清单 46-6 按键中断的服务函数(stm32f4xx_it.c 文件)

```
1
2 void KEY1_IRQHandler(void)
3 {
4     //确保是否产生了 EXTI Line 中断
5     if (EXTI_GetITStatus(KEY1_INT_EXTI_LINE) != RESET) {
6         LED_BLUE;
7         printf("\r\n KEY1 按键中断唤醒 \r\n");
8         EXTI_ClearITPendingBit(KEY1_INT_EXTI_LINE);
9     }
10 }
11
12 void KEY2_IRQHandler(void)
13 {
14     //确保是否产生了 EXTI Line 中断
15     if (EXTI_GetITStatus(KEY2_INT_EXTI_LINE) != RESET) {
16         LED_BLUE;
```

```
17     printf("\r\n KEY2 按键中断唤醒 \r\n");  
18     //清除中断标志位  
19     EXTI_ClearITPendingBit(KEY2_INT_EXTI_LINE);  
20 }  
21 }
```

用于唤醒睡眠模式的中断，其中断服务函数也没有特殊要求，跟普通的应用一样。

46.3.3 下载验证

下载这个实验测试时，可连接上串口，在电脑端的串口调试助手获知调试信息。当系统进入睡眠状态的时候，可以按 KEY1 或 KEY2 按键唤醒系统。

注意：

当系统处于睡眠模式低功耗状态时(包括后面讲解的停止模式及待机模式)，使用 DAP 下载器是无法给芯片下载程序的，所以下载程序时要先把系统唤醒。或者使用如下方法：按着板子的复位按键，使系统处于复位状态，然后点击电脑端的下载按钮下载程序，这时再释放复位按键，就能正常给板子下载程序了。

46.4 PWR—停止模式实验

在睡眠模式实验的基础上，我们进一步讲解如何进入停止模式及唤醒后的状态恢复。

46.4.1 硬件设计

本实验中的硬件与睡眠模式中的一致，主要使用到了按键、LED 彩灯以及使用串口输出调试信息。

46.4.2 软件设计

本小节讲解的是“PWR—停止模式”实验，请打开配套的代码工程阅读理解。

1. 程序设计要点

- (1) 初始化用于唤醒的中断按键；
- (2) 设置停止状态时的 FLASH 供电或掉电；
- (3) 选择电压调节器的工作模式并进入停止状态；
- (4) 使用按键中断唤醒芯片；
- (5) 重启 HSE 时钟，使系统完全恢复停止前的状态。

2. 代码分析

重启 HSE 时钟

与睡眠模式不一样，系统从停止模式被唤醒时，是使用 HSI 作为系统时钟的，在 STM32F407 中，HSI 时钟一般为 16MHz，与我们常用的 168MHz 相关太远，它会影响各种外设的工作频率。所以在系统从停止模式唤醒后，若希望各种外设恢复正常的工作状态，

就要恢复停止模式前使用的系统时钟，本实验中定义了一个 SYSCLKConfig_STOP 函数，用于恢复系统时钟，它的定义见代码清单 25-3。

代码清单 46-7 恢复系统时钟(main.c 文件)

```
1 /**
2  * @brief  停机唤醒后配置系统时钟：使能 HSE, PLL
3  *          并且选择 PLL 作为系统时钟。
4  * @param  None
5  * @retval None
6  */
7 static void SYSCLKConfig_STOP(void)
8 {
9     /* After wake-up from STOP reconfigure the system clock */
10    /* 使能 HSE */
11    RCC_HSEConfig(RCC_HSE_ON);
12
13    /* 等待 HSE 准备就绪 */
14    while (RCC_GetFlagStatus(RCC_FLAG_HSERDY) == RESET);
15
16    /* 使能 PLL */
17    RCC_PLLCmd(ENABLE);
18
19    /* 等待 PLL 准备就绪 */
20    while (RCC_GetFlagStatus(RCC_FLAG_PLLRDY) == RESET);
21
22    /* 选择 PLL 作为系统时钟源 */
23    RCC_SYSCLKConfig(RCC_SYSCLKSource_PLLCLK);
24
25    /* 等待 PLL 被选择为系统时钟源 */
26    while (RCC_GetSYSCLKSource() != 0x08);
27 }
```

这个函数主要是调用了各种 RCC 相关的库函数，开启了 HSE 时钟、使能 PLL 并且选择 PLL 作为时钟源，从而恢复停止前的时钟状态。

main 函数

停止模式实验的 main 函数流程与睡眠模式的类似，主要是调用指令方式的不同及唤醒后增加了恢复时钟的操作，见代码清单 25-2。

代码清单 46-8 停止模式的 main 函数(main.c 文件)

```
1
2 /**
3  * @brief  主函数
4  * @param  无
5  * @retval 无
6  */
7 int main(void)
8 {
9     LED_GPIO_Config();
10
11    /*初始化 USART1*/
12    Debug_USART_Config();
13
14    /* 初始化按键为中断模式，按下中断后会进入中断服务函数 */
15    EXTI_Key_Config();
16
17    printf("\r\n 欢迎使用野火  STM32 F407 开发板。 \r\n");
18    printf("\r\n 野火 F407 停止模式例程 \r\n");
19
20    printf("\r\n 实验说明: \r\n");
```



```
21
22     printf("\r\n 1.本程序中, 绿灯表示 STM32 正常运行, 红灯表示停止状态, 蓝灯表示刚从停止状态被唤醒\r\n");
23     printf("\r\n 2.程序运行一段时间后自动进入停止状态, 在停止状态下, 可使用 KEY1 或 KEY2 唤醒\r\n");
24     printf("\r\n 3.本实验执行这样一个循环: \r\n");
25 printf("\r\n --》亮绿灯 (正常运行)->亮红灯 (停止模式)->按 KEY1 或 KEY2 唤醒->亮蓝灯 (刚被唤醒)---》 \r\n");
26     printf("\r\n 4.在停止状态下, DAP 下载器无法给 STM32 下载程序, \r\n");
27     printf("\r\n 可按 KEY1、KEY2 唤醒后下载, \r\n");
28     printf("\r\n 或按复位键使芯片处于复位状态, 然后在电脑上点击下载按钮, 再释放复位按键, 即可下载\r\n");
29
30     while (1) {
31         /*****执行任务*****/
32         printf("\r\n STM32 正常运行, 亮绿灯\r\n");
33
34         LED_GREEN;
35         Delay(0x3FFFFFFF);
36
37         /*****任务执行完毕, 进入停止降低功耗*****/
38
39         printf("\r\n 进入停止模式, 按 KEY1 或 KEY2 按键可唤醒\r\n");
40
41         //使用红灯指示, 进入停止状态
42         LED_RED;
43
44         /*设置停止模式时, FLASH 进入掉电状态*/
45         PWR_FlashPowerDownCmd (ENABLE);
46         /* 进入停止模式, 设置电压调节器为低功耗模式, 等待中断唤醒 */
47         PWR_EnterSTOPMode(PWR_Regulator_LowPower, PWR_STOPEntry_WFI);
48
49         //等待中断唤醒 K1 或 K2 按键中断
50         /*****被唤醒*****/
51         //获取刚被唤醒时的时钟状态
52         //时钟源
53         clock_source_wakeup = RCC_GetSYSCLKSource ();
54         //时钟频率
55         RCC_GetClocksFreq(&clock_status_wakeup);
56
57         //从停止模式下被唤醒后使用的是 HSI 时钟, 此处重启 HSE 时钟, 使用 PLLCLK
58         SYSCLKConfig_STOP();
59
60         //获取重新配置后的时钟状态
61         //时钟源
62         clock_source_config = RCC_GetSYSCLKSource ();
63         //时钟频率
64         RCC_GetClocksFreq(&clock_status_config);
65
66         //因为刚唤醒的时候使用的是 HSI 时钟, 会影响串口波特率, 输出不对, 所以在重新配置时钟源后才使用串口输出。
67         printf("\r\n 重新配置后的时钟状态: \r\n");
68         printf(" SYSCLK 频率:%d,\r\n HCLK 频率:%d,\r\n PCLK1 频率:%d,\r\n PCLK2 频
           率:%d,\r\n 时钟源:%d (0 表示 HSI, 8 表示 PLLCLK)\n",
           clock_status_config.SYSCLK_Frequency,
           clock_status_config.HCLK_Frequency,
           clock_status_config.PCLK1_Frequency,
           clock_status_config.PCLK2_Frequency,
           clock_source_config);
69
70         printf("\r\n 刚唤醒的时钟状态: \r\n");
71         printf(" SYSCLK 频率:%d,\r\n HCLK 频率:%d,\r\n PCLK1 频率:%d,\r\n PCLK2 频
           率:%d,\r\n 时钟源:%d (0 表示 HSI, 8 表示 PLLCLK)\n",
           clock_status_wakeup.SYSCLK_Frequency,
           clock_status_wakeup.HCLK_Frequency,
           clock_status_wakeup.PCLK1_Frequency,
           clock_status_wakeup.PCLK2_Frequency,
           clock_source_wakeup);
72
73         /*指示灯*/
74
75
76
77
78
79
80
81
82
83
```

```
84     LED_BLUE;  
85     Delay(0x1FFFFFFF);  
86  
87     printf("\r\n 已退出停止模式\r\n");  
88     //继续执行 while 循环  
89 }  
90 }
```

这个 main 函数的执行流程见图 46-5。

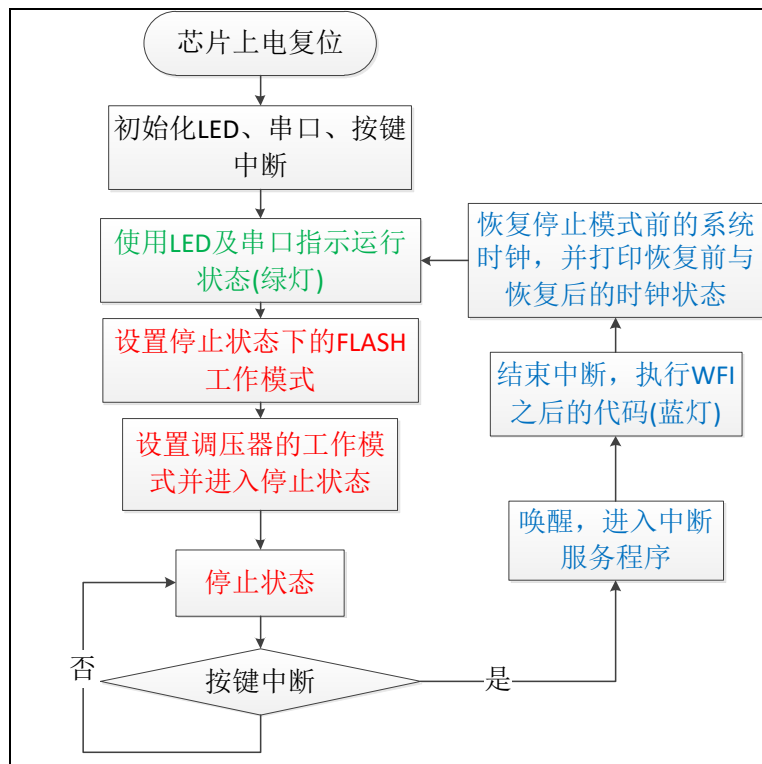


图 46-6 停止模式实验流程图

- (1) 程序中首先初始化了 LED 灯及串口以便用于指示芯片的运行状态，并且把实验板上的两个按键都初始化成了中断模式，以便当系统进入停止模式的时候可以通过按键来唤醒。这些硬件的初始化过程都跟前面章节中的一模一样。
- (2) 初始化完成后使用 LED 及串口表示运行状态，在本实验中，LED 彩灯为绿色时表示正常运行，红灯时表示停止状态，蓝灯时表示刚从停止状态中被唤醒。在停止模式下，I/O 口会保持停止前的状态，所以 LED 彩灯在停止模式时也会保持亮红灯。
- (3) 程序执行一段时间后，我们先用库函数 `PWR_FlashPowerDownCmd` 设置 FLASH 的在停止状态时使用掉电模式，接着调用库函数 `PWR_EnterSTOPMode` 把调压器设置在低功耗模式，并使用 `WFI` 指令进入停止状态。由于 `WFI` 停止模式可以使用任意 `EXTI` 的中断唤醒，所以我们可以使用按键中断唤醒。
- (4) 当系统进入睡眠状态后，我们按下实验板上的 `KEY1` 或 `KEY2` 按键，即可唤醒系统，当执行完中断服务函数后，会继续执行 `WFI` 指令(即 `PWR_EnterSTOPMode` 函数)后的代码。
- (5) 为了更清晰地展示停止模式的影响，在刚唤醒后，我们调用了库函数 `RCC_GetSYSCLKSource` 以及 `RCC_GetClocksFreq` 获取刚唤醒后的系统的时钟源

以及时钟频率，在使用 `SYSClkConfig_STOP` 恢复时钟后，我们再次获取这些时钟状态，最后再通过串口打印出来。

- (6) 通过串口调试信息我们会知道刚唤醒时系统时钟使用的是 `HIS` 时钟，频率为 `16MHz`，恢复后的系统时钟采用 `HSE` 倍频后的 `PLL` 时钟，时钟频率为 `180MHz`。

46.4.3 下载验证

下载这个实验测试时，可连接上串口，在电脑端的串口调试助手获知调试信息。当系统进入停止状态的时候，可以按 `KEY1` 或 `KEY2` 按键唤醒系统。

注意：

当系统处于停止模式低功耗状态时(包括睡眠模式及待机模式)，使用 `DAP` 下载器是无法给芯片下载程序的，所以下载程序时要先把系统唤醒。或者使用如下方法：按着板子的复位按键，使系统处于复位状态，然后点击电脑端的下载按钮下载程序，这时再释放复位按键，就能正常给板子下载程序了。

46.5 PWR—待机模式实验

最后我们来学习最低功耗的待机模式。

46.5.1 硬件设计

本实验中的硬件与睡眠模式、停止模式中的一致，主要使用到了按键、LED 彩灯以及使用串口输出调试信息。要强调的是，由于 `WKUP` 引脚(`PA0`)必须使用上升沿才能唤醒待机状态的系统，所以我们硬件设计的 `PA0` 引脚连接到按键 `KEY1`，且按下按键的时候会在 `PA0` 引脚产生上升沿，从而可实现唤醒的功能，按键的具体电路请查看配套的原理图。

46.5.2 软件设计

本小节讲解的是“PWR—待机模式”实验，请打开配套的代码工程阅读理解。

1. 程序设计要点

- (1) 清除 `WUF` 标志位；
- (2) 使能 `WKUP` 唤醒功能；
- (3) 进入待机状态。

2. 代码分析

main 函数

待机模式实验的执行流程比较简单，见代码清单 25-2。

代码清单 46-9 停止模式的 `main` 函数(`main.c` 文件)

```
1
2 /**
3  * @brief 主函数
```

```
4  * @param 无
5  * @retval 无
6  */
7  int main(void)
8  {
9      LED_GPIO_Config();
10
11     /*初始化 USART1*/
12     Debug_USART_Config();
13
14     /*初始化按键,不需要中断,仅初始化 KEY2 即可,只用于唤醒的 PA0 引脚不需要这样初始化*/
15     Key_GPIO_Config();
16
17     printf("\r\n 欢迎使用野火 STM32 F407 开发板.\r\n");
18     printf("\r\n 野火 F407 待机模式例程\r\n");
19
20     printf("\r\n 实验说明: \r\n");
21
22     printf("1.绿灯表示本次复位是上电或引脚复位,红灯表示即将进入待机状态,蓝灯表示本次是待机唤醒的复位\r\n");
23     printf("\r\n 2.长按 KEY2 按键后,会进入待机模式\r\n");
24     printf("\r\n 3.在待机模式下,按 KEY1 按键可唤醒,唤醒后系统会进行复位,程序从头开始执行\r\n");
25     printf("\r\n 4.可通过检测 WU 标志位确定复位来源\r\n");
26
27     printf("\r\n 5.在待机状态下,DAP 下载器无法给 STM32 下载程序,需要唤醒后才能下载");
28
29     //检测复位来源
30     if (PWR_GetFlagStatus(PWR_FLAG_WU) == SET) {
31         LED_BLUE;
32         printf("\r\n 待机唤醒复位 \r\n");
33     } else {
34         LED_GREEN;
35         printf("\r\n 非待机唤醒复位 \r\n");
36     }
37     while (1) {
38         // K2 按键长按进入待机模式
39         if (KEY2_LongPress()) {
40
41             printf("\r\n 即将进入待机模式,进入待机模式后可按 KEY1 唤醒,唤醒后会进行复位,程序从头开始执行\r\n");
42             LED_RED;
43             Delay(0xFFFFF);
44             /*清除 WU 状态位*/
45             PWR_ClearFlag (PWR_FLAG_WU);
46
47             /* 使能 WKUP 引脚的唤醒功能 , 使能 PA0*/
48             PWR_WakeUpPinCmd (ENABLE);
49
50             /* 进入待机模式 */
51             PWR_EnterSTANDBYMode();
52         }
53     }
54 }
```

这个 main 函数的执行流程见图 46-5。

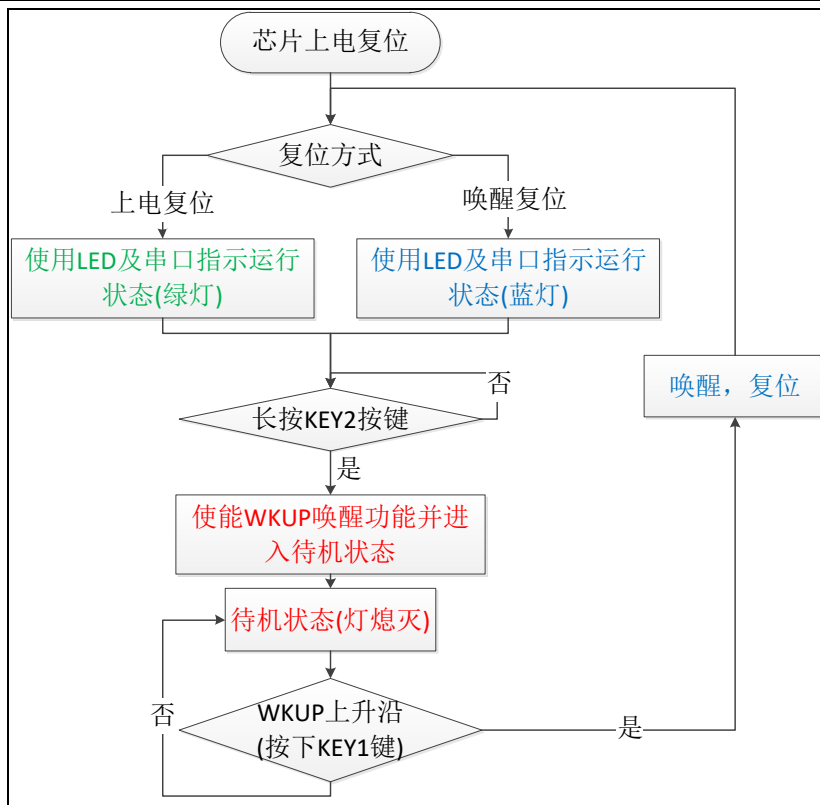


图 46-7 待机模式实验流程图

- (1) 程序中首先初始化了 LED 灯及串口以便用于指示芯片的运行状态，由于待机模式唤醒使用 WKUP 引脚并不需要特别的引脚初始化，所以我们调用的按键初始化函数 Key_GPIO_Config 它的内部只初始化了 KEY2 按键，而且是普通的输入模式，对唤醒用的 PA0 引脚可以不初始化。当然，如果不初始化 PA0 的话，在正常运行模式中 KEY1 按键是不能正常运行的，我们这里只是强调待机模式的 WKUP 唤醒不需要中断，也不需要像按键那样初始化。本工程中使用的 Key_GPIO_Config 函数定义如代码清单 46-10 所示。

代码清单 46-10 Key_GPIO_Config 函数(bsp_key.c 文件)

```
1
2 /**
3  * @brief 配置按键用到的 I/O 口
4  * @param 无
5  * @retval 无
6  */
7 void Key_GPIO_Config(void)
8 {
9     GPIO_InitTypeDef GPIO_InitStructure;
10
11     /*开启按键 GPIO 口的时钟*/
12     RCC_AHB1PeriphClockCmd(KEY2_GPIO_CLK, ENABLE);
13
14     /*设置引脚为输入模式*/
15     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN;
16
17     /*设置引脚不上拉也不下拉*/
18     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
19
20     /*选择按键的引脚*/
```

```
21     GPIO_InitStructure.GPIO_Pin = KEY2_PIN;  
22  
23     /*使用上面的结构体初始化按键*/  
24     GPIO_Init(KEY2_GPIO_PORT, &GPIO_InitStructure);  
25 }
```

- (2) 使用库函数 `PWR_GetFlagStatus` 检测 `PWR_FLAG_WU` 标志位，当这个标志位为 SET 状态的时候，表示本次系统是从待机模式唤醒的复位，否则可能是上电复位。我们利用这个区分两种复位形式，分别使用蓝色 LED 灯或绿色 LED 灯来指示。
- (3) 在 while 循环中，使用自定义的函数 `KEY2_LongPress` 来检测 KEY2 按键是否被长时间按下，若长时间按下则进入待机模式，否则继续 while 循环。`KEY2_LongPress` 函数不是本章分析的重点，感兴趣的读者请自行查阅工程中的代码。
- (4) 检测到 KEY2 按键被长时间按下，要进入待机模式。在使用库函数 `PWR_EnterSTANDBYMode` 发送待机命令前，要先使用库函数 `PWR_ClearFlag` 清除 `PWR_FLAG_WU` 标志位，并且使用库函数 `PWR_WakeUpPinCmd` 使能 WKUP 唤醒功能，这样进入待机模式后才能使用 WKUP 唤醒。
- (5) 在进入待机模式前我们控制了 LED 彩灯为红色，但在待机状态时，由于 I/O 口会处于高阻态，所以 LED 灯会熄灭。
- (6) 按下 KEY1 按键，会使 PA0 引脚产生一个上升沿，从而唤醒系统。
- (7) 系统唤醒后会进行复位，从头开始执行上述过程，与第一次上电时不同的是，这样的复位会使 `PWR_FLAG_WU` 标志位改为 SET 状态，所以这个时候 LED 彩灯会亮蓝色。

46.5.3 下载验证

下载这个实验测试时，可连接上串口，在电脑端的串口调试助手获知调试信息。长按实验板上的 KEY2 按键，系统会进入待机模式，按 KEY1 按键可唤醒系统。

注意：

当系统处于待机模式低功耗状态时(包括睡眠模式及停止模式)，使用 DAP 下载器是无法给芯片下载程序的，所以下载程序时要先把系统唤醒。或者使用如下方法：按着板子的复位按键，使系统处于复位状态，然后点击电脑端的下载按钮下载程序，这时再释放复位按键，就能正常给板子下载程序了。

46.6 PWR—PVD 电源监控实验

这一小节我们学习如何使用 PVD 监控供电电源，增强系统的鲁棒性。

46.6.1 硬件设计

本实验中使用 PVD 监控 STM32 芯片的 VDD 引脚，当监测到供电电压低于阈值时会产生 PVD 中断，系统进入中断服务函数进入紧急处理过程。所以进行这个实验时需要使用一个可调的电压源给实验板供电，改变给 STM32 芯片的供电电压，为此我们需要先了解实验板的电源供电系统，见图 46-8。

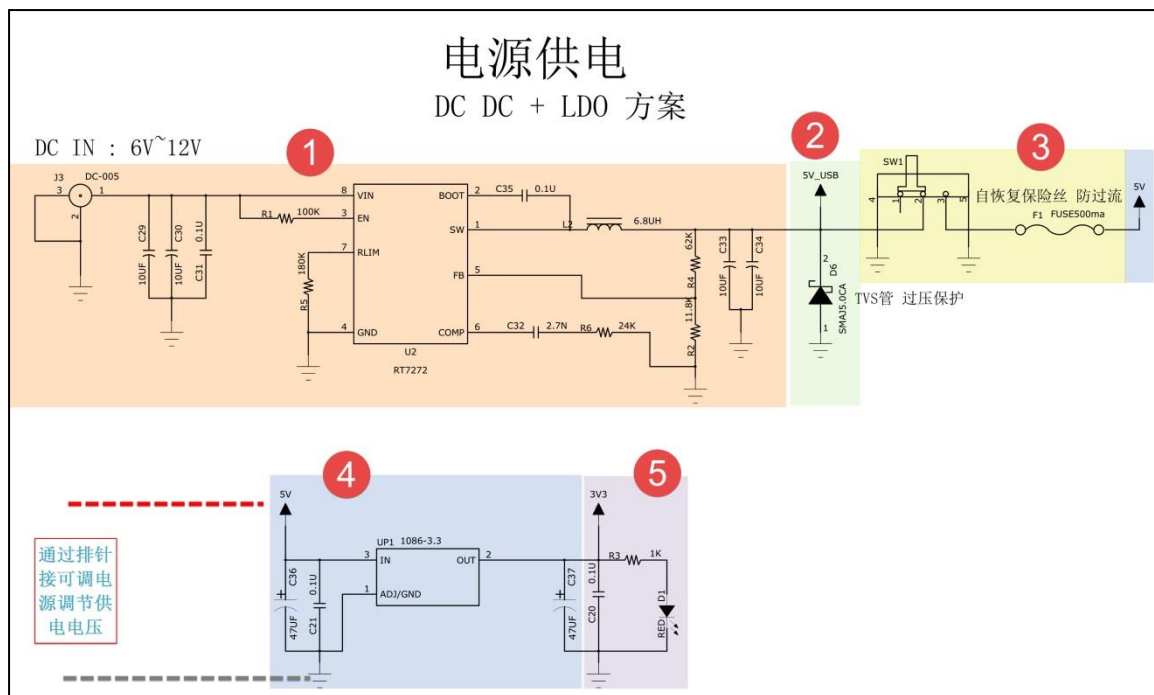


图 46-8 实验板的电源供电系统

整个电源供电系统主要分为以下五部分：

- (1) 6-12V 的 DC 电源供电系统，这部分使用 DC 电源接口引入 6-12V 的电源，经过 RT7272 进行电压转换成 5V 电源，再与第二部分的“5V_USB”电源线连接在一起。
- (2) 第二部分使用 USB 接口，使用 USB 线从外部引入 5V 电源，引入的电源经过电源开关及保险丝连接到“5V”电源线。
- (3) 第三部分的是电源开关及保险丝，即当我们的实验板使用 DC 电源或“5V_USB”线供电时，可用电源开关控制通断，保险丝也会起保护作用。
- (4) “5V”电源线遍布整个板子，板子上各个位置引出的标有“5V”丝印的排针都与这个电源线直接相连。5V 电源线给板子上的某些工作电压为 5V 的芯片供电。5V 电源还经过 LDO 稳压芯片，输出 3.3V 电源连接到“3.3V”电源线。
- (5) 同样地，“3.3V”电源线也遍布整个板子，各个引出的标有“3.3V”丝印的排针都与它直接相连，3.3V 电源给工作电压为 3.3V 的各种芯片供电。STM32 芯片的 VDD 引脚就是直接与这个 3.3V 电源相连的，所以通过 STM32 的 PVD 监控的就是这个“3.3V”电源线的电压。

当我们进行这个 PVD 实验时，为方便改变“3.3V”电源线的电压，我们可以把可调电源通过实验板上引出的“5V”及“GND”排针给实验板供电，当可调电源电压降低时，LDO 在“3.3V”电源线的供电电压会随之降低，即 STM32 的 PVD 监控的 VDD 引脚电压会降低，这样我们就可以模拟 VDD 电压下降的实验条件，对 PVD 进行测试了。不过，由于这样供电不经过保险丝，所以在调节电压的时候要小心，不要给它供电远高于 5V，否则可能会烧坏实验板上的芯片。

46.6.2 软件设计

本小节讲解的是“PWR—睡眠模式”实验，请打开配套的代码工程阅读理解。为了方便把这个工程的 PVD 监控功能移植到其它应用，我们把 PVD 电压监控相关的主要代码都写到“bsp_pvd.c”及“bsp_pvd.h”文件中，这些文件是我们自己编写的，不属于标准库的内容，可根据您的喜好命名文件。

1. 程序设计要点

- (1) 初始化 PVD 中断；
- (2) 设置 PVD 电压监控等级并使能 PVD；
- (3) 编写 PVD 中断服务函数，处理紧急任务。

2. 代码分析

初始化 PVD

使用 PVD 功能前需要先初始化，我们把这部分代码封装到 PVD_Config 函数中，见代码清单 46-11。

代码清单 46-11 初始化 PVD(bsp_pvd.c 文件)

```
1
2 /**
3  * @brief 配置 PVD.
4  * @param None
5  * @retval None
6  */
7 void PVD_Config(void)
8 {
9     NVIC_InitTypeDef NVIC_InitStructure;
10    EXTI_InitTypeDef EXTI_InitStructure;
11
12    /*使能 PWR 时钟 */
13    RCC_APB1PeriphClockCmd(RCC_APB1Periph_PWR, ENABLE);
14
15    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_1);
16
17    /* 使能 PVD 中断 */
18    NVIC_InitStructure.NVIC_IRQChannel = PVD_IRQn;
19    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
20    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0;
21    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
22    NVIC_Init(&NVIC_InitStructure);
23
24    /* 配置 EXTI16 线(PVD 输出) 来产生上升下降沿中断*/
25    EXTI_ClearITPendingBit(EXTI_Line16);
26    EXTI_InitStructure.EXTI_Line = EXTI_Line16;
27    EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
28    EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Rising_Falling;
29    EXTI_InitStructure.EXTI_LineCmd = ENABLE;
30    EXTI_Init(&EXTI_InitStructure);
31
32    //配置 PVD 级别 5
33    // (PVD 检测电压的阈值为 2.8V, VDD 电压低于 2.8V 时产生 PVD 中断,
34    // 具体数据可查询数据手册获知)
35    /*具体级别根据自己的实际应用要求配置*/
```

```
36 PWR_PVDLevelConfig(PWR_PVDLevel_5);
37
38 /* 使能 PVD 输出 */
39 PWR_PVDCmd(ENABLE);
40 }
```

在这段代码中，执行的流程如下：

- (1) 配置 PVD 的中断优先级。由于电压下降是非常危急的状态，所以请尽量把它配置成最高优先级。
- (2) 配置了 EXTI16 线的中断源，设置 EXTI16 是因为 PVD 中断是通过 EXTI16 产生中断的(GPIO 的中断是 EXTI0-EXTI15)。
- (3) 使用库函数 PWR_PVDLevelConfig 设置 PVD 监控的电压阈值等级，各个阈值等级表示的电压值请查阅表 46-2 或 STM32 的数据手册。
- (4) 最后使用库函数 PWR_PVDCmd 使能 PVD 功能。

PVD 中断服务函数

配置完成 PVD 后，还需要编写中断服务函数，在其中处理紧急任务，本工程的 PVD 中断服务函数见代码清单 46-12。

代码清单 46-12 PVD 中断服务函数(stm32f4xx_it.c 文件)

```
1
2 /**
3  * @brief PVD 中断请求
4  * @param None
5  * @retval None
6  */
7 void PVD_IRQHandler(void)
8 {
9     /*检测是否产生了 PVD 警告信号*/
10     if (PWR_GetFlagStatus (PWR_FLAG_PVDO) == SET) {
11         /* 亮红灯，实际应用中应进入紧急状态处理 */
12         LED_RED;
13     }
14     /* 清除中断信号*/
15     EXTI_ClearITPendingBit(EXTI_Line16);
16 }
17
18 }
19
```

注意这个中断服务函数的名是 PVD_IRQHandler 而不是 EXTI16_IRQHandler(STM32 没有这样的中断函数名)，示例中我们仅点亮了 LED 红灯，不同的应用中要根据需求进行相应的紧急处理。

main 函数

本电源监控实验的 main 函数执行流程比较简单，仅调用了 PVD_Config 配置监控功能，当 VDD 供电电压正常时，板子亮绿灯，当电压低于阈值时，会跳转到中断服务函数中，板子亮红灯，见代码清单 25-2。

代码清单 46-13 停止模式的 main 函数(main.c 文件)

```
1 /**
2  * @brief 主函数
3  * @param 无
4  * @retval 无
```

```
5  */
6  int main(void)
7  {
8      LED_GPIO_Config();
9
10     //亮绿灯，表示正常运行
11     LED_GREEN;
12
13     //配置 PVD，当电压过低时，会进入中断服务函数，亮红灯
14     PVD_Config();
15
16     while (1) {
17
18         /*正常运行的程序*/
19
20     }
21 }
22 }
23
24
```

46.6.3 下载验证

本工程的验证步骤如下：

- (1) 通过电脑把本工程编译并下载到实验板；
- (2) 把下载器、USB 及 DC 电源等外部供电设备都拔掉；
- (3) 按“硬件设计”小节中的说明，使用可调电源通过“5V”及“GND”排针给实验板供 5V 电源；(注意要先调好可调电源的电压再连接，防止烧坏实验板)
- (4) 复位实验板，确认板子亮绿灯，表示正常状态；
- (5) 持续降低可调电源的输出电压，直到实验板亮红灯，这时表示 PVD 检测到电压低于阈值。

本工程中，我们实测 PVD 阈值等级为“PWR_PVDLevel_5”时，当可调电源电压降至 4V 时，板子亮红灯，此时的“3.3V”电源引脚的实测电压为 2.8V

注意：

由于这样使用可调电源供电不经过保险丝，所以在调节电压的时候要小心，不要给它供电远高于 5V，否则可能会烧坏实验板上的芯片。
